Micro-Architecture Specification: MIPI DSI Host Controller for Foveated Rendering Systems

Document Version 1.1 ––– Updated with UPF Power Intent and GLS Strategy

MTech VLSI Design ––– Physical Design Project

April 2026

Table of Contents

Micro-Architecture Specification

MIPI DSI Host Controller for Foveated Rendering Systems

Document Version: 1.1 (Updated with UPF Power Intent and GLS Strategy)
Date: April 2026
Status: Approved for RTL Implementation
Target Technology: 45 nm Standard Cell Library
Target Frequency: 200 MHz (byte_clk), 148.5 MHz (pixel_clk)
Power Methodology: IEEE 1801 (UPF) — 3 Power Domains

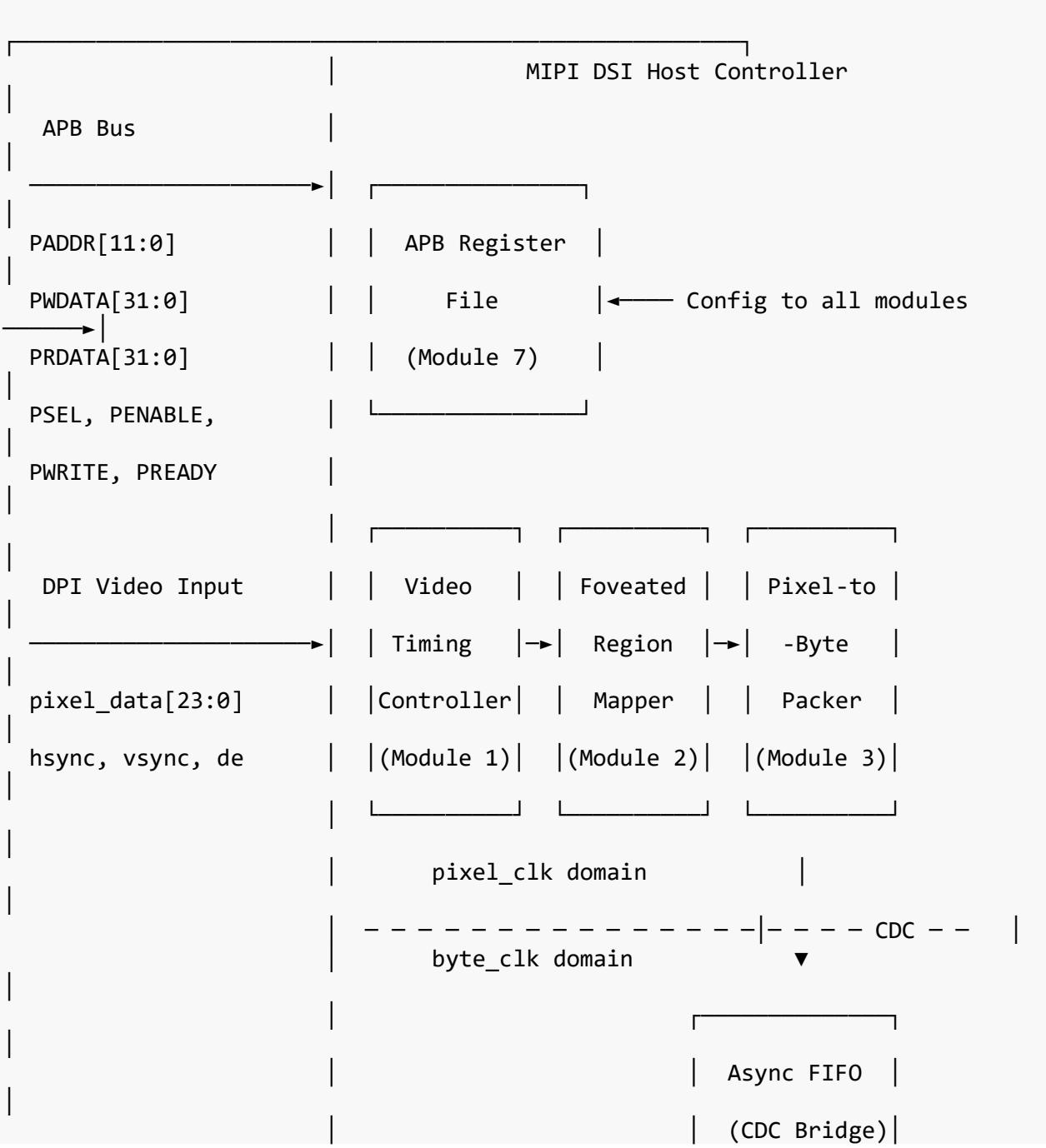
---

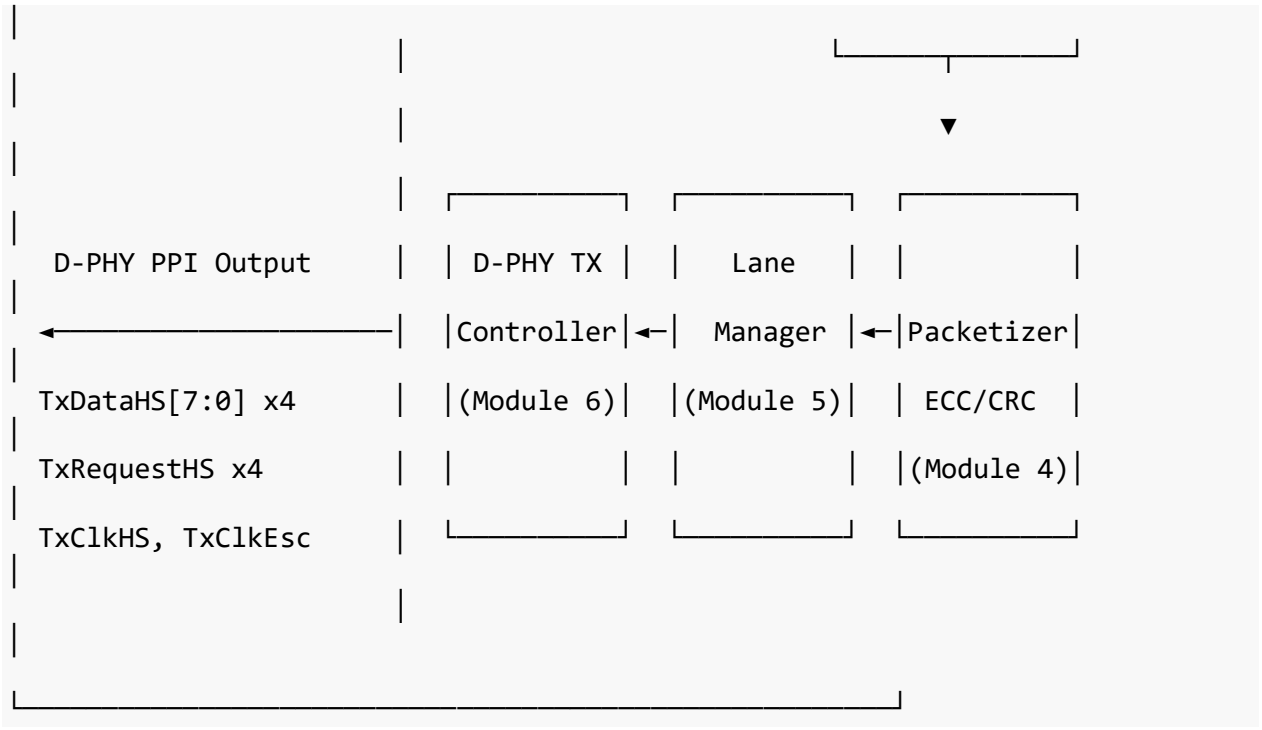
1. Design Overview

1.1 Purpose

This document defines the micro-architecture of a MIPI DSI v1.3-compliant Host Controller with foveated rendering support. It specifies every module's functionality, interfaces, internal state machines, register maps, and design constraints to enable direct RTL coding without ambiguity.

1.2 Top-Level Block Diagram (Text Representation)





1.3 Clock Domains

Clock	Frequency	Domain	Modules
`pixel_clk`	148.5 MHz (1080p/60)	Pixel input processing	Video Timing Controller, Foveated Region Mapper, Pixel-to-Byte Packer
`byte_clk`	200 MHz	DSI protocol + PHY	Packetizer, Lane Manager, D-PHY TX Controller, APB Register File
`apb_clk`	Same as `byte_clk`	Configuration	APB Register File

1.4 Reset Strategy

- Single active-low asynchronous reset `rst_n` with synchronous de-assertion in each clock domain.
- Each clock domain has its own reset synchronizer (2-FF synchronizer on `rst_n`).
- Reset synchronizer module: `reset_sync.v`

```
// Reset synchronizer — instantiate one per clock domain
module reset_sync (
    input  wire clk,
    input  wire rst_n_async,  // Asynchronous active-low reset
    output wire rst_n_sync    // Synchronized reset output
);
    reg [1:0] sync_ff;
    always @(posedge clk or negedge rst_n_async) begin
        if (!rst_n_async)
            sync_ff <= 2'b00;
        else
```

```
            sync_ff <= {sync_ff[0], 1'b1};
    end
    assign rst_n_sync = sync_ff[1];
endmodule
```

---

2. Module 1: Video Timing Controller (`dsi_vtc.v`)

2.1 Functionality

Generates horizontal and vertical timing signals for configurable display resolutions. Accepts DPI-format video input (RGB + sync + data enable) and produces framed pixel data with line/frame markers for downstream processing. This module also generates blanking period indicators used by the Packetizer to insert blanking packets (HSS, HSE, VSS, VSE).

2.2 Interface Signals

Signal	Direction	Width	Clock Domain	Description
`pixel_clk`	Input	1	—	Pixel clock
`rst_n`	Input	1	—	Active-low async reset
`cfg_h_active`	Input	12	`byte_clk`	Horizontal active pixels (e.g., 1920)
`cfg_h_fp`	Input	12	`byte_clk`	Horizontal front porch (pixels)
`cfg_h_sync`	Input	12	`byte_clk`	Horizontal sync width (pixels)
`cfg_h_bp`	Input	12	`byte_clk`	Horizontal back porch (pixels)
`cfg_v_active`	Input	12	`byte_clk`	Vertical active lines (e.g., 1080)
`cfg_v_fp`	Input	12	`byte_clk`	Vertical front porch (lines)
`cfg_v_sync`	Input	12	`byte_clk`	Vertical sync width (lines)
`cfg_v_bp`	Input	12	`byte_clk`	Vertical back porch (lines)
`pixel_data_in[23:0]`	Input	24	`pixel_clk`	RGB888 pixel data from DPI source
`hsync_in`	Input	1	`pixel_clk`	Horizontal sync from DPI
`vsync_in`	Input	1	`pixel_clk`	Vertical sync from DPI
`de_in`	Input	1	`pixel_clk`	Data enable from DPI
`pixel_data_out[23:0]`	Output	24	`pixel_clk`	Pixel data to Foveated Mapper
`pixel_valid`	Output	1	`pixel_clk`	Pixel data valid strobe
`h_count[11:0]`	Output	12	`pixel_clk`	Current horizontal pixel position (0-based)
`v_count[11:0]`	Output	12	`pixel_clk`	Current vertical line position (0-based)

Signal	Direction	Width	Clock Domain	Description
`line_start`	Output	1	`pixel_clk`	Pulse at start of each active line
`line_end`	Output	1	`pixel_clk`	Pulse at end of each active line
`frame_start`	Output	1	`pixel_clk`	Pulse at start of each frame
`frame_end`	Output	1	`pixel_clk`	Pulse at end of each frame
`in_hsync`	Output	1	`pixel_clk`	Currently in horizontal sync period
`in_vsync`	Output	1	`pixel_clk`	Currently in vertical sync period
`in_hbp`	Output	1	`pixel_clk`	Currently in horizontal back porch
`in_hfp`	Output	1	`pixel_clk`	Currently in horizontal front porch

2.3 Internal Design

Two free-running counters: `h_counter` (0 to H_TOTAL-1) and `v_counter` (0 to V_TOTAL-1). The h_counter increments every `pixel_clk` cycle. When h_counter reaches H_TOTAL-1, it resets and v_counter increments.

Timing calculation for 1080p/60fps:

```
H_ACTIVE  = 1920     V_ACTIVE  = 1080
H_FP      = 88       V_FP      = 4
H_SYNC    = 44       V_SYNC    = 5
H_BP      = 148      V_BP      = 36
H_TOTAL   = 2200     V_TOTAL   = 1125
pixel_clk = 2200 × 1125 × 60 = 148.5 MHz
```

2.4 Design Notes

- All `cfg_*` registers are in the `byte_clk` domain. They must be double-registered into `pixel_clk` domain before use. Since these are quasi-static (changed only during blanking or reset), a simple 2-FF synchronizer per bit suffices.
- The VTC can operate in two modes: (a) **Internal timing generation** using counters from config registers, or (b) **External sync passthrough** where it uses `hsync_in`, `vsync_in`, `de_in` directly. Mode selected by `cfg_vtc_mode` register bit.

---

3. Module 2: Foveated Region Mapper (`dsi_fov_mapper.v`)

3.1 Functionality

Takes the current pixel position (h_count, v_count) and gaze fixation coordinates (gaze_x, gaze_y), and computes the eccentricity (distance from gaze point). Based on configurable

radius thresholds, it assigns each pixel to one of three quality tiers. The tier signal travels alongside pixel data through the pipeline, controlling downstream pixel format selection in the Packer.

3.2 Algorithm

The eccentricity is computed as squared Euclidean distance to avoid a square root operation (which is not synthesizable as combinational logic at high speed):

```
dist_sq = (h_count - gaze_x)² + (v_count - gaze_y)²
```

Tier assignment:

```
if      (dist_sq <= radius_foveal_sq)   → tier = 2'b00 (Foveal: full quality)
else if (dist_sq <= radius_mid_sq)      → tier = 2'b01 (Mid-peripheral:
reduced)
else                                    → tier = 2'b10 (Outer-peripheral:
minimum)
```

Implementation detail: The subtraction and squaring use signed arithmetic. (`h_count - gaze_x`) is computed as a 13-bit signed value, squared to produce a 25-bit unsigned result. Two 25-bit squares are summed to produce a 26-bit `dist_sq`. Comparison against radius thresholds (also stored as squared values) is a simple unsigned comparison.

Pipeline: The multiply-and-add requires 2 clock cycles in the pipeline: - Stage 1: Compute `dx = h_count - gaze_x`, `dy = v_count - gaze_y` - Stage 2: Compute `dist_sq = dx*dx + dy*dy`, compare against thresholds

3.3 Interface Signals

Signal	Direction	Width	Clock Domain	Description
`pixel_clk`	Input	1	—	Pixel clock
`rst_n`	Input	1	—	Active-low reset
`pixel_data_in[23:0]`	Input	24	`pixel_clk`	Pixel data from VTC
`pixel_valid_in`	Input	1	`pixel_clk`	Valid strobe from VTC
`h_count[11:0]`	Input	12	`pixel_clk`	Horizontal position
`v_count[11:0]`	Input	12	`pixel_clk`	Vertical position
`cfg_gaze_x[11:0]`	Input	12	`byte_clk`	Gaze X coordinate (synced to pixel_clk internally)
`cfg_gaze_y[11:0]`	Input	12	`byte_clk`	Gaze Y coordinate
`cfg_radius_foveal_sq[25:0]`	Input	26	`byte_clk`	Foveal radius^2 threshold
`cfg_radius_mid_sq[25:0]`	Input	26	`byte_clk`	Mid-peripheral radius^2 threshold
`cfg_fov_enable`	Input	1	`byte_clk`	Enable foveation (0 = all

Signal	Direction	Width	Clock Domain	Description
				Tier 0)
`pixel_data_out[23:0]`	Output	24	`pixel_clk`	Pixel data (delayed by 2 cycles)
`pixel_valid_out`	Output	1	`pixel_clk`	Valid strobe (delayed by 2 cycles)
`fov_tier[1:0]`	Output	2	`pixel_clk`	Quality tier: 00=foveal, 01=mid, 10=outer

3.4 Bypass Mode

When `cfg_fov_enable = 0`, the mapper outputs `fov_tier = 2'b00` (foveal) for all pixels, effectively disabling foveation. This is the baseline mode for power/bandwidth comparison measurements.

3.5 Design Notes

- The 13-bit × 13-bit multiplier will be inferred as a single-cycle combinational multiplier by Genus at 200 MHz. If timing fails, insert a pipeline register between multiply and add.
- Gaze coordinates are updated by software through APB registers. They are quasi-static (updated once per frame at ~16 ms intervals), so 2-FF synchronization from `byte_clk` to `pixel_clk` is safe.
- For a 1080p display: `gaze_x` range is 0–1919, `gaze_y` range is 0–1079.
- Typical foveal radius: ~100 pixels (5° eccentricity at typical VR viewing distance). Radius^2 = 10000. Mid radius: ~300 pixels (20°). Radius^2 = 90000.

---

4. Module 3: Pixel-to-Byte Packer (`dsi_packer.v`)

4.1 Functionality

Converts RGB pixel data into a byte stream according to the assigned foveation tier. Each tier maps to a different DSI pixel format, reducing bandwidth for peripheral regions.

4.2 Pixel Format Mapping by Tier

Tier	Pixel Format	DSI Data Type (DT)	Bytes/Pixel	Bandwidth vs. Full
00 (Foveal)	RGB888 (24-bit)	`0x3E`	3.0	100%
01 (Mid)	RGB666 Packed (18-bit)	`0x1E`	2.25	75%
10 (Outer)	RGB565 (16-bit)	`0x0E`	2.0	67%

4.3 Packing Details

RGB888 (Tier 0): Each pixel produces exactly 3 bytes: `{R[7:0], G[7:0], B[7:0]}`. Straightforward, no packing complexity.

RGB666 Packed (Tier 1): Three 18-bit pixels pack into 6.75 bytes. Per DSI spec, 4 pixels pack into 9 bytes:

```
Pixel 0: R0[5:0] G0[5:0] B0[5:0]  (18 bits)
Pixel 1: R1[5:0] G1[5:0] B1[5:0]  (18 bits)
Pixel 2: R2[5:0] G2[5:0] B2[5:0]  (18 bits)
Pixel 3: R3[5:0] G3[5:0] B3[5:0]  (18 bits)
Total: 72 bits = 9 bytes
```

RGB565 (Tier 2): Each pixel produces exactly 2 bytes: `{R[4:0], G[5:0], B[4:0]}` → `{byte0: R[4:0]|G[5:3], byte1: G[2:0]|B[4:0]}`.

4.4 Interface Signals

Signal	Direction	Width	Clock Domain	Description
`pixel_clk`	Input	1	—	Pixel clock
`rst_n`	Input	1	—	Active-low reset
`pixel_data_in[23:0]`	Input	24	`pixel_clk`	Pixel data from Foveated Mapper
`pixel_valid_in`	Input	1	`pixel_clk`	Pixel valid
`fov_tier[1:0]`	Input	2	`pixel_clk`	Quality tier
`line_start`	Input	1	`pixel_clk`	Start of active line
`line_end`	Input	1	`pixel_clk`	End of active line
`byte_data[7:0]`	Output	8	`pixel_clk`	Packed byte output
`byte_valid`	Output	1	`pixel_clk`	Byte output valid
`byte_sof`	Output	1	`pixel_clk`	Start of frame marker
`byte_sol`	Output	1	`pixel_clk`	Start of line marker
`byte_eol`	Output	1	`pixel_clk`	End of line marker
`byte_dt[5:0]`	Output	6	`pixel_clk`	DSI Data Type for current line segment
`byte_wc[15:0]`	Output	16	`pixel_clk`	Word count (payload bytes) for current line

4.5 Design Notes

- **Simplification for initial implementation:** For the first RTL iteration, implement uniform tier per line (the tier of the first pixel on each line determines the format for the entire line). This avoids mid-line format switching complexity. The foveated mapper produces horizontal "bands" naturally since vertical gaze position determines which lines are foveal.

- In a later revision, per-pixel tier switching can be implemented using DSI's multi-packet-per-line feature (multiple long packets per line with different DTs).
- The packer must count the number of payload bytes per line segment to generate `byte_wc` for the Packetizer's long packet header.

---

5. Module 4: Packetizer with ECC/CRC (`dsi_packetizer.v`)

5.1 Functionality

Forms complete DSI packets (short and long) from the byte stream. Generates packet headers with ECC protection and appends CRC to long packet payloads. Produces the complete packet byte sequence that feeds the Lane Manager.

5.2 DSI Packet Structures

Short Packet (4 bytes, no payload):

```
Byte 0: DI[7:0] = {VC[1:0], DT[5:0]}   – Data Identifier
Byte 1: Data0[7:0]                       – First data byte
Byte 2: Data1[7:0]                       – Second data byte
Byte 3: ECC[7:0]                         – Error Correction Code (Hamming
8,6)
```

Used for: VSS (DT=0x01), VSE (DT=0x11), HSS (DT=0x21), HSE (DT=0x31), and other sync events.

Long Packet (4 + WC + 2 bytes):

```
Byte 0:      DI[7:0] = {VC[1:0], DT[5:0]}     – Data Identifier
Byte 1:      WC_LSB[7:0]                        – Word Count (payload length)
LSB
Byte 2:      WC_MSB[7:0]                        – Word Count MSB
Byte 3:      ECC[7:0]                            – ECC over bytes 0-2
Byte 4..N+3: Payload[0..N-1]                     – Pixel data (N = WC)
Byte N+4:    CRC_LSB[7:0]                        – CRC-16 LSB
Byte N+5:    CRC_MSB[7:0]                        – CRC-16 MSB
```

Used for: Packed Pixel Stream (DT=0x0E/0x1E/0x3E for RGB565/RGB666/RGB888).

5.3 ECC Algorithm (Hamming 8,6)

The ECC protects the 3-byte packet header (24 bits: DI + WC/Data). It can correct single-bit errors and detect 2-bit errors.

```
// ECC generation for 24-bit header {DI, WC_LSB/Data0, WC_MSB/Data1}
// Input: D[23:0] = {header_byte2, header_byte1, header_byte0}
function [7:0] calc_ecc;
    input [23:0] D;
    begin
        calc_ecc[0] =
```

```
D[0]^D[1]^D[2]^D[4]^D[5]^D[7]^D[10]^D[11]^D[13]^D[16]^D[20]^D[21]^D[22]^D[23]
;
        calc_ecc[1] =
D[0]^D[1]^D[3]^D[4]^D[6]^D[8]^D[10]^D[12]^D[14]^D[17]^D[20]^D[21]^D[22]^D[23]
;
        calc_ecc[2] =
D[0]^D[2]^D[3]^D[5]^D[6]^D[9]^D[11]^D[12]^D[15]^D[18]^D[20]^D[21]^D[22];
        calc_ecc[3] =
D[1]^D[2]^D[3]^D[7]^D[8]^D[9]^D[13]^D[14]^D[15]^D[19]^D[20]^D[21]^D[23];
        calc_ecc[4] =
D[4]^D[5]^D[6]^D[7]^D[8]^D[9]^D[16]^D[17]^D[18]^D[19]^D[20]^D[22]^D[23];
        calc_ecc[5] =
D[10]^D[11]^D[12]^D[13]^D[14]^D[15]^D[16]^D[17]^D[18]^D[19]^D[21]^D[22]^D[23]
;
        calc_ecc[6] = 1'b0;
        calc_ecc[7] = 1'b0;
    end
endfunction
```

5.4 CRC-16 Algorithm

Polynomial: $x^{16} + x^{12} + x^5 + 1$ (0x8408, bit-reversed representation). Initial seed: 0xFFFF.

```
// CRC-16 computation — process one byte per clock
function [15:0] crc16_byte;
    input [15:0] crc_prev;
    input [7:0]  data_byte;
    reg [15:0] crc;
    integer i;
    begin
        crc = crc_prev;
        for (i = 0; i < 8; i = i + 1) begin
            if ((crc[0] ^ data_byte[i]) == 1'b1)
                crc = {1'b0, crc[15:1]} ^ 16'h8408;
            else
                crc = {1'b0, crc[15:1]};
        end
        crc16_byte = crc;
    end
endfunction
```

5.5 Packetizer State Machine

```
States:
  IDLE        → Waiting for data or sync event
  SHORT_PKT   → Outputting 4-byte short packet (sync events)
  LONG_HDR    → Outputting 4-byte long packet header
  LONG_DATA   → Outputting payload bytes (pixel data), computing CRC
  LONG_CRC    → Outputting 2-byte CRC footer
  BLANKING    → Outputting blanking packet during non-active periods
```

```
Transitions:
  IDLE       → SHORT_PKT  : when vsync/hsync event detected
  IDLE       → LONG_HDR   : when line pixel data available
  SHORT_PKT  → IDLE       : after 4 bytes output
  LONG_HDR   → LONG_DATA  : after 4 bytes output
  LONG_DATA  → LONG_CRC   : after WC bytes output
  LONG_CRC   → IDLE       : after 2 bytes output
```

5.6 Interface Signals

Signal	Direction	Width	Clock Domain	Description
`byte_clk`	Input	1	—	Byte clock
`rst_n`	Input	1	—	Active-low reset
`fifo_data[7:0]`	Input	8	`byte_clk`	Pixel byte data from CDC FIFO
`fifo_valid`	Input	1	`byte_clk`	FIFO data valid
`fifo_sof`	Input	1	`byte_clk`	Start of frame
`fifo_sol`	Input	1	`byte_clk`	Start of line
`fifo_eol`	Input	1	`byte_clk`	End of line
`fifo_dt[5:0]`	Input	6	`byte_clk`	Data type for current segment
`fifo_wc[15:0]`	Input	16	`byte_clk`	Word count for current segment
`fifo_rd_en`	Output	1	`byte_clk`	FIFO read enable
`cfg_vc[1:0]`	Input	2	`byte_clk`	Virtual Channel ID
`cfg_video_mode[1:0]`	Input	2	`byte_clk`	00=Non-Burst Sync Pulse, 01=Non-Burst Sync Event, 10=Burst
`pkt_data[7:0]`	Output	8	`byte_clk`	Packet byte to Lane Manager
`pkt_valid`	Output	1	`byte_clk`	Packet byte valid
`pkt_sop`	Output	1	`byte_clk`	Start of packet
`pkt_eop`	Output	1	`byte_clk`	End of packet
`pkt_type`	Output	1	`byte_clk`	0=short pkt, 1=long pkt

5.7 Video Mode Packet Sequence (Non-Burst Sync Pulse)

Each line in a frame follows this sequence:

```
Active line:   HSS → [HSA payload] → HSE → [HBP payload] → RGB pixel long pkt
→ [HFP payload]
Vsync line:    VSS → HSS → [HSA] → HSE → [HBP blanking] → [HFP blanking]
VBP line:      HSS → [HSA] → HSE → [HBP blanking] → [HFP blanking]
VFP line:      HSS → [HSA] → HSE → [HBP blanking] → [HFP blanking]
```

For initial implementation, use **Non-Burst Sync Event** mode (simplest):

```
Active line:   HSS → RGB pixel long packet → blanking
Vsync line:    VSS → blanking
```

---

6. Module 5: Lane Manager (`dsi_lane_mgr.v`)

6.1 Functionality

Distributes packet bytes from the Packetizer across 1, 2, or 4 data lanes according to the DSI interleaving scheme. Also handles the Low-Power Data Transmission (LPDT) escape mode for command-mode operations.

6.2 Lane Distribution Algorithm

Bytes from the Packetizer are distributed in round-robin across active lanes:

```
1-lane mode:  All bytes → Lane 0
2-lane mode:  Byte 0 → Lane 0, Byte 1 → Lane 1, Byte 2 → Lane 0, ...
4-lane mode:  Byte 0 → Lane 0, Byte 1 → Lane 1, Byte 2 → Lane 2, Byte 3 →
Lane 3, ...
```

Important: The first byte of every packet always goes to Lane 0. Between packets, the lane counter resets to 0. This means short packets (4 bytes) use all 4 lanes in one cycle in 4-lane mode, or take 4 cycles in 1-lane mode.

6.3 Interface Signals

Signal	Direction	Width	Clock Domain	Description
`byte_clk`	Input	1	—	Byte clock
`rst_n`	Input	1	—	Active-low reset
`pkt_data[7:0]`	Input	8	`byte_clk`	Packet byte from Packetizer
`pkt_valid`	Input	1	`byte_clk`	Packet byte valid
`pkt_sop`	Input	1	`byte_clk`	Start of packet (resets lane counter)
`pkt_eop`	Input	1	`byte_clk`	End of packet
`cfg_lane_count[1:0]`	Input	2	`byte_clk`	00=1 lane, 01=2 lanes, 10=4 lanes
`lane0_data[7:0]`	Output	8	`byte_clk`	Lane 0 byte data
`lane0_valid`	Output	1	`byte_clk`	Lane 0 data valid
`lane1_data[7:0]`	Output	8	`byte_clk`	Lane 1 byte data
`lane1_valid`	Output	1	`byte_clk`	Lane 1 data valid
`lane2_data[7:0]`	Output	8	`byte_clk`	Lane 2 byte data
`lane2_valid`	Output	1	`byte_clk`	Lane 2 data valid
`lane3_data[7:0]`	Output	8	`byte_clk`	Lane 3 byte data

Signal	Direction	Width	Clock Domain	Description
`lane3_valid`	Output	1	`byte_clk`	Lane 3 data valid
`hs_request`	Output	1	`byte_clk`	Request HS mode to D-PHY TX
`hs_active`	Input	1	`byte_clk`	D-PHY TX in HS mode, ready for data

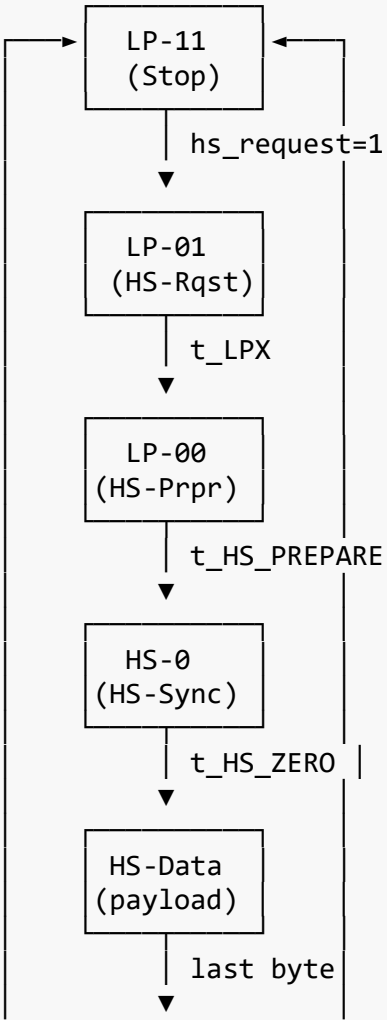
---

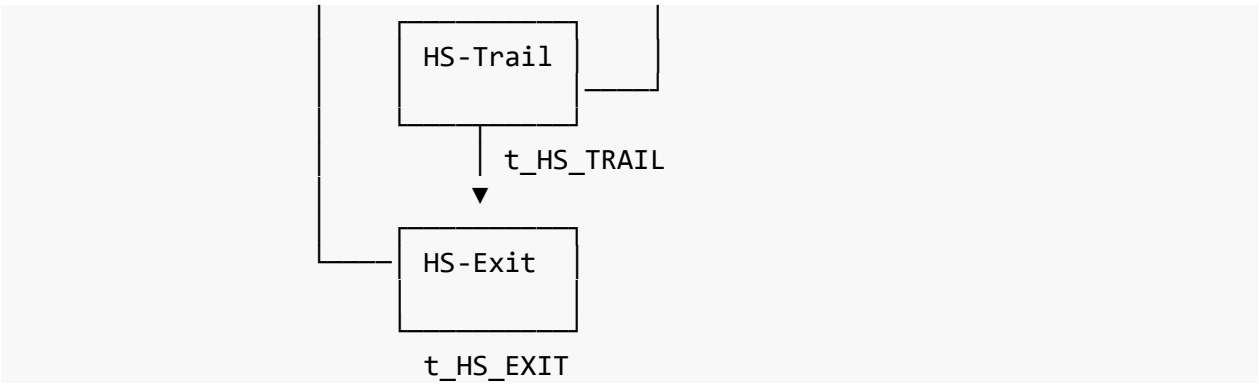
7. Module 6: D-PHY TX Controller (`dsi_dphy_tx.v`)

7.1 Functionality

Implements the D-PHY digital transmit interface at the PPI (PHY-Protocol Interface) level. Manages High-Speed and Low-Power state machine transitions for data lanes and clock lane. This module does NOT implement analog drivers — it outputs PPI-level control signals that would connect to an analog D-PHY macro in a real SoC.

7.2 D-PHY Lane State Machine (Per Data Lane)





7.3 Timing Parameters (Configurable via registers)

Parameter	Min (ns)	Typical (byte_clk cycles @ 200MHz)	Description
t_LPX	50	10	LP-01 duration
t_HS_PREPARE	40+4×UI	9	LP-00 duration before HS
t_HS_ZERO	105+6×UI	22	HS-0 duration
t_HS_TRAIL	max(8×UI, 60+4×UI)	13	Post-data trail
t_HS_EXIT	100	20	Return to LP-11
t_CLK_PREPARE	38	8	Clock lane prepare
t_CLK_ZERO	300-t_CLK_PREPARE	54	Clock lane zero
t_CLK_PRE	8×UI	2	Clock before data
t_CLK_POST	60+52×UI	16	Clock after data
t_CLK_TRAIL	60	12	Clock trail

Where UI (Unit Interval) = 1 / (2 × bitrate_per_lane). At 800 Mbps: UI = 1.25 ns.

7.4 Clock Lane Controller

The clock lane operates independently. In **continuous clock mode** (simplest for implementation), the clock lane stays in HS mode whenever the controller is active, providing a continuous DDR clock to the receiver.

```
States: CLK_LP11 → CLK_HS_RQST → CLK_HS_PRPR → CLK_HS_ZERO → CLK_HS_RUN →
CLK_HS_TRAIL → CLK_HS_EXIT → CLK_LP11
```

7.5 PPI Interface Signals (Per Lane)

Signal	Direction	Width	Description
TxDataHS[7:0]	Output	8	HS transmit data (one byte per byte_clk)
TxRequestHS	Output	1	Request High-Speed transmission
TxReadyHS	Input	1	PHY ready for HS data (from analog PHY)

Signal	Direction	Width	Description
TxDataLP[1:0]	Output	2	LP mode data {Dp, Dn}
TxRequestLP	Output	1	Request LP transmission
TxClkHS	Output	1	Clock lane HS request
TxClkLP[1:0]	Output	2	Clock lane LP data
TxClkEsc	Output	1	Escape mode clock

7.6 Full Interface Table

Signal	Direction	Width	Clock Domain	Description
byte_clk	Input	1	—	Byte clock (200 MHz)
rst_n	Input	1	—	Active-low reset
lane0_data[7:0]	Input	8	byte_clk	Data lane 0 from Lane Manager
lane0_valid	Input	1	byte_clk	Lane 0 valid
lane1_data[7:0]	Input	8	byte_clk	Data lane 1
lane1_valid	Input	1	byte_clk	Lane 1 valid
lane2_data[7:0]	Input	8	byte_clk	Data lane 2
lane2_valid	Input	1	byte_clk	Lane 2 valid
lane3_data[7:0]	Input	8	byte_clk	Data lane 3
lane3_valid	Input	1	byte_clk	Lane 3 valid
hs_request	Input	1	byte_clk	Request HS mode from Lane Manager
hs_active	Output	1	byte_clk	HS mode active, data lanes ready
cfg_cont_clk	Input	1	byte_clk	1=continuous clock, 0=non-continuous
cfg_t_lpx[7:0]	Input	8	byte_clk	t_LPX in byte_clk cycles
cfg_t_hs_prepare[7:0]	Input	8	byte_clk	t_HS_PREPARE in cycles
cfg_t_hs_zero[7:0]	Input	8	byte_clk	t_HS_ZERO in cycles
cfg_t_hs_trail[7:0]	Input	8	byte_clk	t_HS_TRAIL in cycles
cfg_t_hs_exit[7:0]	Input	8	byte_clk	t_HS_EXIT in cycles
cfg_t_clk_prepare[7:0]	Input	8	byte_clk	CLK t_CLK_PREPARE
cfg_t_clk_zero[7:0]	Input	8	byte_clk	CLK t_CLK_ZERO
cfg_t_clk_post[7:0]	Input	8	byte_clk	CLK t_CLK_POST
cfg_t_clk_trail[7:0]	Input	8	byte_clk	CLK t_CLK_TRAIL
TxDataHS_0[7:0] … TxDataHS_3[7:0]	Output	8 each	byte_clk	PPI HS data per lane

Signal	Direction	Width	Clock Domain	Description
TxRequestHS_0 … TxRequestHS_3	Output	1 each	byte_clk	PPI HS request per lane
TxDataLP_0[1:0] … TxDataLP_3[1:0]	Output	2 each	byte_clk	PPI LP data per lane
TxClkHS	Output	1	byte_clk	Clock lane HS enable
TxClkLP[1:0]	Output	2	byte_clk	Clock lane LP state

8. Module 7: APB Register File (dsi_apb_regs.v)

8.1 Functionality

AMBA APB3 slave interface providing read/write access to all configuration and status registers. All registers are in the byte_clk domain.

8.2 APB Interface Signals

Signal	Direction	Width	Description
PCLK	Input	1	APB clock (same as byte_clk)
PRESETn	Input	1	APB active-low reset
PADDR[11:0]	Input	12	Address bus (4KB address space)
PSEL	Input	1	Peripheral select
PENABLE	Input	1	Enable phase
PWRITE	Input	1	Write/Read select
PWDATA[31:0]	Input	32	Write data
PRDATA[31:0]	Output	32	Read data
PREADY	Output	1	Ready (always 1, no wait states)

8.3 Complete Register Map

Offset	Name	R/W	Reset	Description
General Control				
0x000	DSI_CTRL	R/W	0x0000_0000	Main control register
0x004	DSI_STATUS	R	0x0000_0000	Status register
0x008	DSI_INT_STATUS	R/W1C	0x0000_0000	Interrupt status (write 1 to clear)
0x00C	DSI_INT_ENABLE	R/W	0x0000_0000	Interrupt enable mask

Offset	Name	R/W	Reset	Description
Video Timing				
0x010	DSI_VID_HSIZE	R/W	0x0000_0000	{4'b0, h_bp[11:0], 4'b0, h_active[11:0]}
0x014	DSI_VID_HSYNC	R/W	0x0000_0000	{4'b0, h_fp[11:0], 4'b0, h_sync[11:0]}
0x018	DSI_VID_VSIZE	R/W	0x0000_0000	{4'b0, v_bp[11:0], 4'b0, v_active[11:0]}
0x01C	DSI_VID_VSYNC	R/W	0x0000_0000	{4'b0, v_fp[11:0], 4'b0, v_sync[11:0]}
Foveation Control				
0x020	DSI_FOV_CTRL	R/W	0x0000_0000	{31'b0, fov_enable}
0x024	DSI_FOV_GAZE	R/W	0x0000_0000	{4'b0, gaze_y[11:0], 4'b0, gaze_x[11:0]}
0x028	DSI_FOV_RAD_FOVEAL	R/W	0x0000_2710	{6'b0, radius_foveal_sq[25:0]} (default=10000)
0x02C	DSI_FOV_RAD_MID	R/W	0x0001_5F90	{6'b0, radius_mid_sq[25:0]} (default=90000)
Lane & PHY Configuration				
0x030	DSI_PHY_CTRL	R/W	0x0000_0002	{29'b0, cont_clk, lane_count[1:0]}
0x034	DSI_PHY_TMR1	R/W	0x0016_0A09	{t_hs_zero[7:0], t_hs_prepare[7:0], t_lpx[7:0], 8'b0}
0x038	DSI_PHY_TMR2	R/W	0x0014_0D00	{t_hs_exit[7:0], t_hs_trail[7:0], 16'b0}
0x03C	DSI_PHY_CLK_TMR	R/W	0x0C103608	{t_clk_trail[7:0], t_clk_post[7:0], t_clk_zero[7:0], t_clk_prepare[7:0]}
Packet Configuration				
0x040	DSI_PKT_CTRL	R/W	0x0000_0000	{28'b0, vc[1:0], video_mode[1:0]}

Offset	Name	R/W	Reset	Description
Debug / Status				
0x080	`DSI_DBG_FIFO`	R	—	`{16'b0, fifo_wr_level[7:0], fifo_rd_level[7:0]}`
0x084	`DSI_DBG_PKT_CNT`	R	—	Total packets sent counter
0x088	`DSI_DBG_BYTE_CNT`	R	—	Total bytes sent counter
0x08C	`DSI_DBG_FRAME_CNT`	R	—	Frame counter

8.4 DSI_CTRL Register Fields (0x000)

Bits	Name	R/W	Reset	Description
[0]	`dsi_en`	R/W	0	Global DSI controller enable
[1]	`video_en`	R/W	0	Video mode enable (start streaming)
[2]	`cmd_en`	R/W	0	Command mode enable
[3]	`vtc_mode`	R/W	0	0=external sync, 1=internal timing gen
[7:4]	Reserved	R	0	—
[8]	`soft_reset`	R/W	0	Software reset (self-clearing)
[31:9]	Reserved	R	0	—

8.5 DSI_STATUS Register Fields (0x004)

Bits	Name	R	Description
[0]	`phy_ready`	R	D-PHY in HS mode and ready
[1]	`fifo_empty`	R	CDC FIFO empty
[2]	`fifo_full`	R	CDC FIFO full
[3]	`video_active`	R	Currently streaming video data
[4]	`in_vsync`	R	In vertical sync period

---

9. CDC Asynchronous FIFO (`dsi_async_fifo.v`)

9.1 Functionality

Gray-code pointer asynchronous FIFO bridging the `pixel_clk` write domain and `byte_clk` read domain. This is a standard design pattern — use a proven implementation.

9.2 Parameters

Parameter	Value	Rationale
DATA_WIDTH	30 bits	8-bit data + 1 valid + 1 sof + 1 sol + 1 eol + 6-bit DT + 12-bit reserved = 30 bits (round to 32)
FIFO_DEPTH	256	Sufficient to absorb clock frequency difference and line

Parameter	Value	Rationale
	entries	buffering
ADDR_WIDTH	8 bits	log2(256) = 8

9.3 Interface

Signal	Direction	Width	Clock Domain	Description
`wr_clk`	Input	1	—	Write clock (pixel_clk)
`wr_rst_n`	Input	1	—	Write domain reset
`wr_en`	Input	1	`wr_clk`	Write enable
`wr_data[31:0]`	Input	32	`wr_clk`	Write data
`wr_full`	Output	1	`wr_clk`	FIFO full
`wr_level[8:0]`	Output	9	`wr_clk`	Write side fill level
`rd_clk`	Input	1	—	Read clock (byte_clk)
`rd_rst_n`	Input	1	—	Read domain reset
`rd_en`	Input	1	`rd_clk`	Read enable
`rd_data[31:0]`	Output	32	`rd_clk`	Read data
`rd_empty`	Output	1	`rd_clk`	FIFO empty
`rd_level[8:0]`	Output	9	`rd_clk`	Read side fill level

9.4 FIFO Rate Analysis and Depth Justification

The write clock (pixel_clk = 148.5 MHz) is **slower** than the read clock (byte_clk = 200 MHz). This means the read side (consumer) outpaces the write side (producer) in terms of clock cycles per second. The FIFO will tend to run near-empty, not near-full. It will **never overflow** from a steady-state rate mismatch. This is the opposite of the classic "FIFO overflow" problem where writers outpace readers.

Why a FIFO is still necessary despite the reader being faster:

1. **Asynchronous clocks (CDC):** The two clocks come from different PLLs with no guaranteed phase relationship. Even if they were identical frequencies, a flip-flop output changing on `pixel_clk` could violate setup/hold of a `byte_clk`-clocked flip-flop, causing metastability. The Gray-code async FIFO handles this safely.

2. **Bursty consumption pattern:** The Packetizer on the read side does NOT read at a steady rate. It pauses for 4 byte_clk cycles to insert each packet header (DI + WC + ECC), pauses for 2 cycles to insert CRC at end of each long packet, and pauses during sync packet insertion (HSS, VSS, HSE, VSE). During these pauses, the write side keeps writing at 148.5 MHz. The FIFO absorbs these bursts.

Depth calculation:

The longest write-side burst without any read occurs when the Packetizer is assembling and sending blanking/sync packets between lines. The worst case is the end-of-line to

start-of-next-line transition, where the Packetizer sends: HSE (4B short pkt) + HFP blanking (variable) + HSS (4B short pkt) + HSA blanking + HSE + HBP blanking. During this time, the write side may have already started writing the next line's pixel data. The maximum pause duration is approximately 40–60 byte_clk cycles. At 148.5 MHz write rate, approximately 45–67 words accumulate. A depth of 256 entries provides comfortable 4× margin for this worst case, ensuring the FIFO never backpressures the pixel source under any operating condition.

A depth of 16–32 entries would suffice for pure CDC between these frequencies, but 256 handles the bursty consumption pattern with zero risk of overflow.

9.5 Key Design Rules

- Write pointer and read pointer are maintained as Gray-coded values.
- Gray-coded pointers are synchronized across clock domains using 2-FF synchronizers.
- Full condition: compared in write domain. Empty condition: compared in read domain.
- The FIFO word packs byte_data, control flags, and data type into a single 32-bit entry.

FIFO word format:

```
Bit [7:0]   : byte_data[7:0]     – Pixel byte
Bit [8]     : byte_sol            – Start of line
Bit [9]     : byte_eol            – End of line
Bit [10]    : byte_sof            – Start of frame
Bit [16:11] : byte_dt[5:0]        – DSI data type
Bit [31:17] : byte_wc[14:0]       – Word count (first word of line only)
```

---

10. Top-Level Module (`dsi_host_top.v`)

10.1 Top-Level Ports

Port	Direction	Width	Description
Clocks and Reset			
`pixel_clk`	Input	1	Pixel clock (148.5 MHz)
`byte_clk`	Input	1	Byte clock (200 MHz)
`rst_n`	Input	1	Global active-low async reset
DPI Video Input			
`dpi_pdata[23:0]`	Input	24	RGB888 pixel data
`dpi_hsync`	Input	1	Horizontal sync
`dpi_vsync`	Input	1	Vertical sync
`dpi_de`	Input	1	Data enable
APB Configuration			

Port	Direction	Width	Description
PADDR[11:0]	Input	12	APB address
PSEL	Input	1	Peripheral select
PENABLE	Input	1	APB enable
PWRITE	Input	1	Write select
PWDATA[31:0]	Input	32	Write data
PRDATA[31:0]	Output	32	Read data
PREADY	Output	1	APB ready
D-PHY PPI Output			
phy_txdatahs_0[7:0]	Output	8	Lane 0 HS data
phy_txdatahs_1[7:0]	Output	8	Lane 1 HS data
phy_txdatahs_2[7:0]	Output	8	Lane 2 HS data
phy_txdatahs_3[7:0]	Output	8	Lane 3 HS data
phy_txrequesths[3:0]	Output	4	Per-lane HS request
phy_txreadyhs[3:0]	Input	4	Per-lane HS ready (from PHY)
phy_txdatalp_0[1:0]	Output	2	Lane 0 LP data
phy_txdatalp_1[1:0]	Output	2	Lane 1 LP data
phy_txdatalp_2[1:0]	Output	2	Lane 2 LP data
phy_txdatalp_3[1:0]	Output	2	Lane 3 LP data
phy_txclkhs	Output	1	Clock lane HS
phy_txclklp[1:0]	Output	2	Clock lane LP
Interrupts			
dsi_irq	Output	1	Interrupt output

10.2 Estimated Gate Count Breakdown

Module	Estimated Gates	Notes
Video Timing Controller	~2,000	Counters and comparators
Foveated Region Mapper	~4,000	Two 13×13 multipliers + comparators
Pixel-to-Byte Packer	~3,000	Barrel shifter + format muxing
Packetizer (ECC/CRC)	~5,000	CRC-16 LFSR + ECC logic + FSM
Lane Manager	~2,000	Mux + round-robin counter
D-PHY TX Controller	~8,000	5 FSMs (4 data + 1 clock) + timers
APB Register File	~4,000	~20 registers × 32 bits
Async FIFO (256×32)	~12,000	Dual-port SRAM + Gray-code logic
Total	**~40,000**	Well within 45nm capacity

11. Design Constraints Summary

11.1 Synthesis Constraints

- Target frequency: 200 MHz (byte_clk), 148.5 MHz (pixel_clk)
- Clock uncertainty: 0.2 ns (setup), 0.05 ns (hold)
- Max transition: 0.3 ns
- Max fanout: 32
- Input delay: 2 ns (all inputs relative to respective clocks)
- Output delay: 2 ns (all outputs relative to byte_clk)

11.2 Coding Guidelines

- No latches (complete if/else and case statements with defaults)
- Synchronous reset preferred, async reset only on power-on
- All FFs in a module clocked by the same edge (posedge)
- No combinational loops
- No multi-driven signals
- Use `parameter` and `localparam` for all magic numbers
- Use `generate` for replicated lane logic
- Naming convention: `cfg_*` for configuration, `dbg_*` for debug, `int_*` for internal

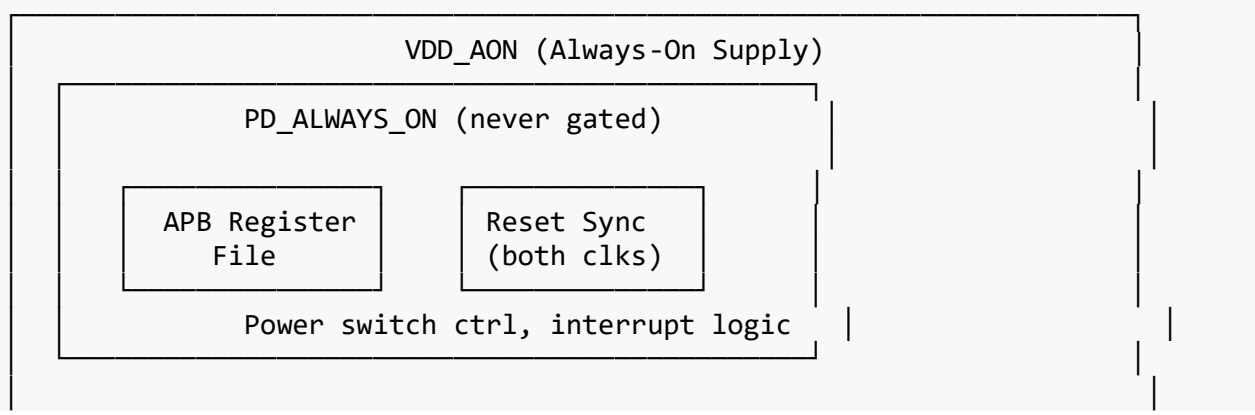
---

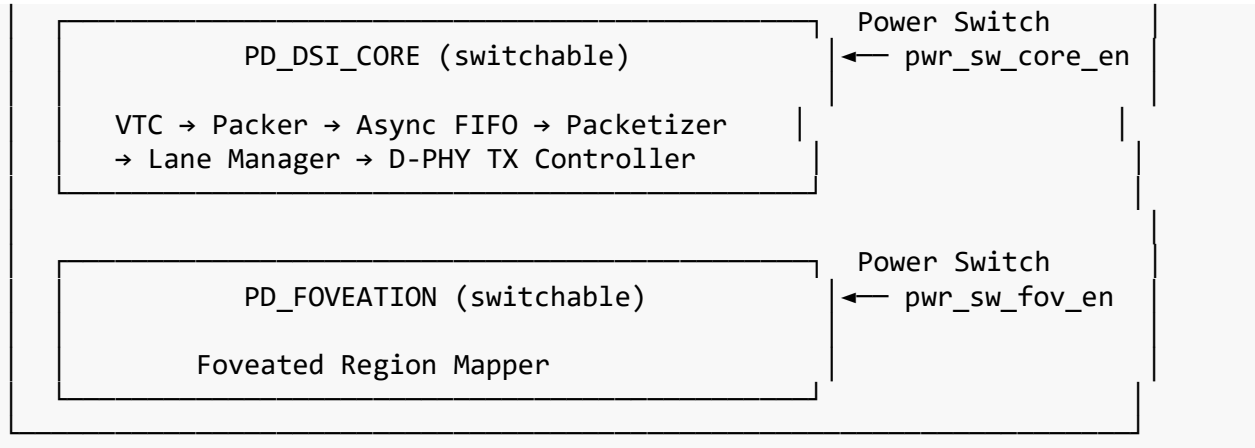
12. Power Domain Architecture (IEEE 1801 UPF)

12.1 Power Intent Overview

The MIPI DSI Host Controller defines three power domains motivated by real-world operating scenarios in mobile and XR SoCs. When a smartphone screen is off, the entire DSI pipeline should be power-gated to eliminate leakage. When the screen is in always-on display mode (showing a clock), only command-mode access is needed, so the video pipeline can be shut down. When foveated rendering is not in use (standard mobile display mode), the Foveated Region Mapper wastes leakage power doing nothing but passing data through. These scenarios map directly to three independently switchable power domains.

12.2 Power Domain Definitions





Power Domain	Supply	Modules	Switchable	Rationale
PD_ALWAYS_ON	VDD_AON	APB Register File, Reset Synchronizers, Interrupt logic, Power switch control	No	Processor must always access config registers and control power switches
PD_DSI_CORE	VDD_CORE (switched from VDD_AON)	VTC, Packer, Async FIFO, Packetizer, Lane Manager, D-PHY TX Controller	Yes, via pwr_sw_core_en	Entire video pipeline shut down when display is off
PD_FOVEATION	VDD_FOV (switched from VDD_AON)	Foveated Region Mapper	Yes, via pwr_sw_fov_en	Foveation is optional; standard display mode does not need this module

12.3 Power State Table

State	PD_ALWAYS_ON	PD_DSI_CORE	PD_FOVEATION	Use Case
S0: Full Off	ON	OFF	OFF	Screen off, deep sleep
S1: Core Active	ON	ON	OFF	Standard display (no foveation)
S2: Foveated Active	ON	ON	ON	VR/AR with eye tracking active
S3: Retention	ON	RETENTION	OFF	Quick-resume display standby

12.4 Isolation Cells

Isolation cells are required at every signal crossing from a switchable domain (that can be powered off) to any domain that remains powered. When a domain is off, its outputs

become undefined (X in simulation). Isolation cells clamp these signals to a safe known value to prevent X-propagation and corruption.

Signal Path	From Domain	To Domain	Isolation Type	Clamp Value	Rationale
`cfg_*` feedback (status)	`PD_DSI_CORE` →	`PD_ALWAYS_ON`	AND-type (clamp to 0)	0	Status bits must read 0 when core is off
`dsi_irq` (from core)	`PD_DSI_CORE` →	`PD_ALWAYS_ON`	AND-type (clamp to 0)	0	No spurious interrupts when off
`phy_txdatahs_*`	`PD_DSI_CORE` →	Primary outputs	AND-type (clamp to 0)	0	D-PHY pins must be low (LP-11 idle) when off
`phy_txrequesths_*`	`PD_DSI_CORE` →	Primary outputs	AND-type (clamp to 0)	0	No HS requests when off
`phy_txdatalp_*`	`PD_DSI_CORE` →	Primary outputs	OR-type (clamp to 1)	`2'b11`	LP-11 is the D-PHY idle state (both lines high)
`fov_tier[1:0]`	`PD_FOVEATION` →	`PD_DSI_CORE`	AND-type (clamp to 0)	`2'b00`	When foveation off, tier=00 means full quality (correct bypass behavior)
`pixel_data_out` (from mapper)	`PD_FOVEATION` →	`PD_DSI_CORE`	Feedthrough (clamp to 0)	0	Pixel data must not be X
`pixel_valid_out` (from mapper)	`PD_FOVEATION` →	`PD_DSI_CORE`	AND-type (clamp to 0)	0	No valid strobe when off

Important design note for the Foveation domain isolation: When `PD_FOVEATION` is off, `fov_tier` is clamped to `2'b00` (foveal/full quality). This means the downstream Packer automatically selects RGB888 mode — which is exactly the correct behavior for non-foveated display mode. The `pixel_data` path, however, requires special handling: when the

mapper is off, pixel data must bypass directly from VTC to Packer. This is achieved by adding a MUX in PD_DSI_CORE controlled by cfg_fov_enable (from always-on APB register): when cfg_fov_enable=0, pixel data routes directly from VTC to Packer, completely bypassing the off mapper. The isolation cells on pixel_data_out and pixel_valid_out from the mapper are then just safety guards.

12.5 Retention Registers

Retention registers have a shadow latch powered by the always-on supply (VDD_AON). Before power-down, a SAVE signal captures the register value into the shadow latch. After power-up, a RESTORE signal loads the shadow value back into the main flip-flop. This avoids the need for software to reprogram configuration after a power-down/up cycle.

Register Group	Domain	Retain?	Rationale
Video timing config (h_active, h_fp, etc.)	PD_DSI_CORE	**Yes**	Avoid software reprogramming 8 timing registers on wake-up
D-PHY timing config (t_lpx, t_hs_prepare, etc.)	PD_DSI_CORE	**Yes**	PHY timing must be exact; re-init is error-prone
Lane count, video mode	PD_DSI_CORE	**Yes**	Basic operating mode preserved
FSM state registers	PD_DSI_CORE	No	FSMs restart from IDLE on power-up; retaining mid-state would cause protocol violations
Counters (h_counter, v_counter, pkt_cnt, etc.)	PD_DSI_CORE	No	Transient values; restart from 0
FIFO pointers	PD_DSI_CORE	No	FIFO must be empty/reset on power-up
Gaze coordinates	PD_FOVEATION	No	Updated every frame by software; no value in retaining
Foveation radius thresholds	PD_FOVEATION	No	Quasi-static but stored in always-on APB registers, not in the mapper

Retention strategy summary: Only configuration registers inside PD_DSI_CORE that would require software re-initialization are retained. This is approximately 12 registers × 32 bits = 384 retention flops. FSM state, counters, and FIFO pointers restart cleanly from reset.

12.6 Power Switches

Each switchable domain has a header-type power switch (PMOS transistor between VDD_AON and the local switched supply). The power switch is controlled by an enable signal from the always-on domain.

Switch	Controls	Enable Signal	Source
PSW_CORE	VDD_CORE for	pwr_sw_core_en	APB register DSI_PWR_CTRL[0] in

Switch	Controls	Enable Signal	Source
	PD_DSI_CORE		PD_ALWAYS_ON
PSW_FOV	VDD_FOV for PD_FOVEATION	pwr_sw_fov_en	APB register DSI_PWR_CTRL[1] in PD_ALWAYS_ON

Power-up sequence: 1. Software writes DSI_PWR_CTRL[0] = 1 to enable PD_DSI_CORE 2. Wait for supply ramp-up (implementation-dependent, typically 1–10 µs) 3. Assert RESTORE to load retained register values 4. De-assert reset to PD_DSI_CORE modules 5. If foveation needed: write DSI_PWR_CTRL[1] = 1, wait for ramp-up, de-assert foveation reset 6. Software writes DSI_CTRL.dsi_en = 1 to start operation

Power-down sequence: 1. Software writes DSI_CTRL.dsi_en = 0, wait for current frame to complete 2. Assert SAVE to capture retention register values 3. Assert reset to domain 4. Write DSI_PWR_CTRL[0] = 0 to disable power switch

12.7 New APB Registers for Power Management

These registers are added to the always-on APB register file:

Offset	Name	R/W	Reset	Description
0x090	DSI_PWR_CTRL	R/W	0x0000_0000	{30'b0, pwr_sw_fov_en, pwr_sw_core_en}
0x094	DSI_PWR_STATUS	R	0x0000_0000	{30'b0, fov_pwr_ack, core_pwr_ack}
0x098	DSI_RET_CTRL	R/W	0x0000_0000	{30'b0, fov_restore, core_restore} — self-clearing

12.8 UPF File Placement in Design Flow

The UPF file is written alongside RTL and SDC, before synthesis. The flow uses it at three stages:

1. **Genus Synthesis:** Reads UPF to insert isolation cells, level shifters, retention flops, and power switch cells from the standard cell library. The synthesized netlist contains these power-management cells.
2. **Innovus Place and Route:** Reads UPF to place power switches in physical rows, route switched power rings/stripes, and ensure isolation cells are placed at domain boundaries.
3. **Power-Aware Simulation (optional):** UPF-aware simulators (Xcelium with +upf flag, or Synopsys VCS with -upf) can simulate power-on/off behavior, verifying that isolation clamps correctly and retention save/restore works.

---

13. Gate-Level Simulation (GLS) Strategy

13.1 Purpose and Placement in Flow

Gate-Level Simulation verifies that the synthesized and physically implemented netlist maintains functional correctness under real gate delays. GLS catches bugs that RTL simulation cannot detect because RTL simulation operates in zero-delay mode where every flip-flop captures data instantly.

GLS is performed at two checkpoints in the physical design flow:

GLS #1 — Post-Synthesis (Week 5): Uses the gate-level netlist from Genus with estimated timing (SDF from synthesis). This catches synthesis interpretation mismatches (unexpected latch inference, incorrect operator inference, constant propagation errors) early before committing to physical design effort.

GLS #2 — Post-Route Signoff (Week 7): Uses the final gate-level netlist from Innovus with extracted parasitics (SPEF → SDF with accurate RC delays). This is the signoff GLS that catches timing-related functional failures caused by actual wire delays, crosstalk, and clock skew from CTS. In industry DV roles, this is the netlist the PD team hands to DV for final sign-off.

13.2 What GLS Catches That RTL Cannot

Timing-related functional failures: A combinational path in the CRC-16 computation that takes 5.2 ns at the worst corner will cause the downstream register to capture stale data at 200 MHz (5.0 ns period). RTL simulation shows correct behavior because it models zero delay. Post-route GLS with SDF annotation will show incorrect CRC values, exposing the timing violation as a functional bug.

Synthesis interpretation mismatches: If a `case` statement in the Packetizer FSM is missing a `default` clause, Genus may infer a latch instead of a mux. RTL simulation might still pass because the simulator assigns X to unhandled cases (which may not trigger assertion failures). GLS will show the latch's actual behavior — holding the previous value — which may cause incorrect packet formation.

CDC metastability behavior: The Gray-code pointers crossing between `pixel_clk` and `byte_clk` through 2-FF synchronizers will show occasional X values or glitches in GLS due to setup/hold violations on the first synchronizer FF. This is **expected and correct** — the second FF resolves the metastability. But testbench assertions must be written to tolerate these X values on intermediate synchronizer nodes. This is a very common DV interview question.

Reset tree skew: In RTL, `rst_n` de-asserts simultaneously for all flops. In GLS, the reset buffer tree has real delays, so different flops see reset de-assertion at slightly different times. If any design logic depends on multiple flops exiting reset simultaneously (which it shouldn't, but bugs happen), GLS will expose it.

13.3 GLS Test Strategy

Full 1080p frame simulation at gate level is impractical — it could take hours to days. The strategy is to create a **dedicated GLS testcase** with reduced parameters:

Parameter	RTL Full Test	GLS Test
Resolution	1920×1080	64×48
Frame count	10+ frames	2–3 frames
Foveation tiers	All 3 tiers	All 3 tiers (verified in small frame)
Lane configs	1, 2, 4 lanes	4-lane only (primary config)
Video modes	All 3 sub-modes	Non-Burst Sync Event only
Simulation time	~5 min (RTL)	~2–4 hours (GLS)
SDF annotation	None	Max delay (setup), Min delay (hold)

The GLS testbench reuses the RTL testbench infrastructure but with these modifications:

1. **Add `$sdf_annotate`** to load the SDF file onto the gate-level netlist instance.
2. **Disable timing checks initially** with `+notimingcheck` to verify functional correctness first, then re-enable for timing verification.
3. **Add X-tolerance** to assertions: the output checker must mask X values during the first few cycles after power-on and during CDC synchronizer settling (typically 2–3 byte_clk cycles after any FIFO write).
4. **Use `+no_notifier`** to suppress setup/hold violation messages from standard cells during expected transitions (clock domain crossing FFs).
5. **Reduce simulation verbosity** — turn off per-packet logging, only check final frame pixel sequence match.

13.4 GLS Invocation Commands

```
# Post-Synthesis GLS (with Genus SDF)
xrun -v93 \
    -v /path/to/stdcell_library.v \       # Standard cell Verilog models
    ../rtl/tb/dsi_host_gls_tb.v \         # GLS testbench
    ../synth/output/dsi_host_top_syn.v \   # Synthesized netlist from Genus
    +define+GLS +define+SDF_ANNOTATE \
    +access+r +notimingcheck \             # First run: functional only
    -sdf_cmd_file sdf_cmd.txt              # SDF annotation commands

# sdf_cmd.txt contents:
# COMPILED sdf_file=../synth/output/dsi_host_top_syn.sdf,
scope=dsi_host_gls_tb.u_dut,
#          log_file=sdf_annotate.log, mtm_spec=maximum

# Post-Route GLS (with Innovus SDF — signoff run)
xrun -v93 \
    -v /path/to/stdcell_library.v \
    ../rtl/tb/dsi_host_gls_tb.v \
```

```
    ../pnr/output/dsi_host_top_route.v \   # Post-route netlist from Innovus
    +define+GLS +define+SDF_ANNOTATE \
    +access+r \                            # Enable timing checks for signoff
    -sdf_cmd_file sdf_cmd_postroute.txt    # Post-route SDF
```

13.5 GLS Testbench Modification Pattern

```
// In the GLS testbench wrapper:
module dsi_host_gls_tb;

    // ... clock and reset generation (same as RTL TB) ...

    // DUT instantiation — gate-level netlist instead of RTL
    dsi_host_top u_dut (
        .pixel_clk    (pixel_clk),
        .byte_clk     (byte_clk),
        .rst_n        (rst_n),
        // ... all ports same as RTL TB ...
    );

    // SDF annotation
    `ifdef SDF_ANNOTATE
    initial begin
        $sdf_annotate("../pnr/output/dsi_host_top_route.sdf", u_dut, ,
"sdf.log", "MAXIMUM");
    end
    `endif

    // X-tolerance for output checking
    // During CDC settling (first 3 byte_clk cycles after FIFO write),
    // mask X values in output comparison
    reg [2:0] cdc_settle_cnt;
    wire      cdc_settled = (cdc_settle_cnt == 3'd0);

    always @(posedge byte_clk or negedge rst_n) begin
        if (!rst_n)
            cdc_settle_cnt <= 3'd3;
        else if (cdc_settle_cnt != 3'd0)
            cdc_settle_cnt <= cdc_settle_cnt - 1;
    end

    // Output checker with X-masking
    always @(posedge byte_clk) begin
        if (cdc_settled && expected_valid) begin
            if (u_dut.phy_txdatahs_0 !== 8'hxx &&
                u_dut.phy_txdatahs_0 !== expected_data) begin
                $error("GLS mismatch: expected=%h, got=%h at time %t",
                       expected_data, u_dut.phy_txdatahs_0, $time);
            end
        end
```

```
    end

endmodule
```

13.6 Expected GLS Results and Deliverables

The GLS deliverable for the final report should include:

1. **Functional pass confirmation:** "Post-route GLS with SDF annotation at maximum delay corner confirmed correct DSI packet output for a 64×48/3-frame test pattern at 200 MHz byte_clk."
2. **Setup/hold violation summary:** Number of violations reported, categorized as CDC-expected (synchronizer FFs — safe to waive) vs. functional-path (must be fixed in PD).
3. **Simulation runtime comparison:** RTL sim time vs. GLS time for the same testcase, demonstrating understanding of the performance tradeoff.
4. **X-propagation analysis:** Document any unexpected X values in functional outputs and their root cause (ideally none after proper isolation and reset design).

Appendix A: DSI Data Type Reference

Data Type	Code	Description
V Sync Start	0x01	Short packet
V Sync End	0x11	Short packet
H Sync Start	0x21	Short packet
H Sync End	0x31	Short packet
End of Transmission	0x08	Short packet
Color Mode Off	0x02	Short packet
Color Mode On	0x12	Short packet
Blanking Packet	0x19	Long packet
Packed Pixel RGB565	0x0E	Long packet
Packed Pixel RGB666	0x1E	Long packet
Packed Pixel RGB888	0x3E	Long packet
DCS Short Write (no param)	0x05	Short packet
DCS Short Write (1 param)	0x15	Short packet
DCS Long Write	0x39	Long packet
DCS Read	0x06	Short packet

Appendix B: RTL and Design File List

File	Module/Purpose	Lines (est.)

File	Module/Purpose	Lines (est.)
reset_sync.v	Reset synchronizer	15
dsi_vtc.v	Video Timing Controller	120
dsi_fov_mapper.v	Foveated Region Mapper	100
dsi_packer.v	Pixel-to-Byte Packer	200
dsi_ecc.v	ECC generator function	30
dsi_crc16.v	CRC-16 generator	40
dsi_packetizer.v	Packetizer FSM	300
dsi_lane_mgr.v	Lane Manager	100
dsi_dphy_tx.v	D-PHY TX Controller	400
dsi_apb_regs.v	APB Register File	200
dsi_async_fifo.v	Asynchronous FIFO	150
dsi_host_top.v	Top-level integration	200
RTL Total		**~1,855**
Design Collateral		
dsi_host_constraints.sdc	Timing constraints for Genus/Innovus/Tempus	160
dsi_host_power.upf	IEEE 1801 power intent (3 domains, isolation, retention)	120
dsi_host_gls_tb.v	Gate-level simulation testbench wrapper	150
sdf_cmd_syn.txt	SDF annotation commands for post-synthesis GLS	5
sdf_cmd_route.txt	SDF annotation commands for post-route GLS	5